

Sales Tracker Go-Live Guide

How To Deploy Sales Tracker From Your Laptop To A Live Website

A simple, detailed guide for uploading your code to GitHub, deploying the frontend to Vercel, deploying the backend to Render, connecting Supabase and Paystack, and checking that everything works.

Prepared for the current Sales Tracker setup: HTML/CSS/JS frontend, Supabase database/auth, Paystack payments, and an Express backend.

Important idea: your website has two live parts. Vercel hosts the frontend pages. Render hosts the backend API. Supabase hosts the database and authentication. Paystack sends payment updates to the backend webhook.

1. Understand The Final Live Setup

Frontend

Hosted on Vercel.

`https://use-sales-tracker.vercel.app`

This is what users open in their browser.

Backend

Hosted on Render.

`https://sales-tracker-app-cd7k.onrender.com`

This handles team routes, subscriptions, Paystack verification, and webhooks.

When the frontend needs protected team features, it calls the backend URL. The backend then talks securely to Supabase using the service role key.

2. Prepare Your Project Folder

Before uploading to GitHub, your project folder should contain the important files below:

```
server.js
package.json
package-lock.json
admin.html
dashboard.html
pricing.html
signin.html
admin.css
dashboard.css
```

```
signin.css
support-widget.js
support-widget.css
img/
```

Do not upload secret keys inside normal HTML files. Your Supabase anon key can be public, but your Supabase service role key and Paystack secret key must only exist in Render environment variables.

Check that `package.json` has this:

```
{
  "type": "module",
  "scripts": {
    "start": "node server.js",
    "dev": "node server.js"
  }
}
```

Expected outcome: running `npm start` or `npm run dev` starts the backend locally.

3. Test Locally Before Uploading

1. Open PowerShell in your project folder.
2. Run `npm install`.
3. Run `npm run dev`.
4. Open `http://localhost:4242/api/health`.

Expected outcome:

```
{"ok":true}
```

If you see `MODULE_NOT_FOUND`, it usually means `server.js` is missing or you are in the wrong folder.

If you see missing environment variables, check your local `.env` file.

4. Create A GitHub Repository

1. Go to `https://github.com`.
2. Click **New repository**.
3. Name it something like `sales-tracker`.
4. Choose Private if you do not want people seeing your code.
5. Do not add a README if your folder already has files.
6. Click **Create repository**.

Expected outcome: GitHub shows you a page with commands for uploading your existing folder.

5. Upload Your Project To GitHub

In PowerShell, inside your project folder, run:

```
git init
git add .
git commit -m "Initial Sales Tracker deployment"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git
git push -u origin main
```

If Git says the remote already exists, use:

```
git remote set-url origin https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git
git push -u origin main
```

Expected outcome: your project files appear inside the GitHub repository.

6. Deploy The Frontend To Vercel

1. Go to `https://vercel.com`.
2. Click **Add New Project**.
3. Import your GitHub repository.
4. If your files are in the repository root, leave Root Directory empty.
5. Framework preset can be **Other** or static site.
6. Build command can be empty for static HTML.
7. Output directory can be empty or root, depending on Vercel prompt.
8. Click **Deploy**.

Expected outcome: Vercel gives you a live URL like:

```
https://use-sales-tracker.vercel.app
```

If Vercel says it cannot find a build output, the easiest fix is usually to keep the frontend as simple static files and make sure `index.html` exists in the deployed root.

7. Deploy The Backend To Render

1. Go to `https://render.com`.
2. Click **New** then **Web Service**.
3. Connect the same GitHub repository.
4. Choose the branch `main`.
5. Set Runtime to Node.
6. Set Build Command to `npm install`.
7. Set Start Command to `npm start`.
8. Click **Create Web Service**.

Expected outcome: Render deploys and gives you a backend URL like:

```
https://sales-tracker-app-cd7k.onrender.com
```

Test it by opening:

```
https://sales-tracker-app-cd7k.onrender.com/api/health
```

Expected result:

```
{"ok":true}
```

8. Add Render Environment Variables

In Render, open your backend service, then go to **Environment**. Add:

Name	Example value	Meaning
APP_URL	https://use-sales-tracker.vercel.app	Your Vercel frontend URL.
SUPABASE_URL	https://your-project.supabase.co	Your Supabase project URL.
SUPABASE_SERVICE_ROLE_KEY	Your real service role key	Secret backend key. Never place in frontend HTML.
PAYSTACK_SECRET_KEY	sk_live_or_test...	Your Paystack secret key.

Common mistake: using the Supabase anon key instead of the service role key. Team creation, subscription activation, and protected backend tasks may fail if the service role key is wrong.

After saving variables, redeploy the backend.

9. Connect Frontend To The Live Backend

In the frontend files, replace local backend URLs with the Render URL.

In `admin.html` and `dashboard.html`:

```
const TEAM_API_URL = "https://sales-tracker-app-cd7k.onrender.com";
```

In `pricing.html`:

```
const PAYMENT_API_URL = "https://sales-tracker-app-cd7k.onrender.com";
```

Expected outcome: your live Vercel website calls Render instead of `localhost:4242`.

10. Set CORS Correctly

Your backend should allow your Vercel frontend URL. In `server.js`, keep an allowed origins list like:

```
const allowedOrigins = [  
  appUrl,  
  "https://use-sales-tracker.vercel.app",  
  "http://localhost:5500",  
  "http://127.0.0.1:5500",  
  "http://127.0.0.1:3000"  
];
```

Do not add a final slash to the Vercel origin. Use this:

```
https://use-sales-tracker.vercel.app
```

Not this:

```
https://use-sales-tracker.vercel.app/
```

11. Set Paystack Webhook

In Paystack dashboard, set your webhook URL to:

```
https://sales-tracker-app-cd7k.onrender.com/api/paystack/webhook
```

Expected outcome: after payment, Paystack sends the transaction result to your backend, and the backend activates the user's subscription/business access in Supabase.

12. Supabase Setup Checklist

- ☐ Tables exist: businesses, profiles, customers, products.
- ☐ profiles has business_id and role.
- ☐ customers has business_id and user_id.
- ☐ products has business_id.
- ☐ Admin profile exists and has role admin.
- ☐ Business row exists or backend can create it.
- ☐ Subscription status is active for users who should access the dashboard.

If the dashboard says a business column does not exist, run the SQL repair/upgrade scripts in Supabase SQL editor.

13. Final Test After Deployment

1. Open `https://sales-tracker-app-cd7k.onrender.com/api/health`.
2. Confirm you see `{"ok":true}`.
3. Open `https://use-sales-tracker.vercel.app`.

4. Sign in as admin.
5. Open Admin Dashboard.
6. Add a product.
7. Add a rep.
8. Open Sales Dashboard as rep.
9. Add a sale.
10. Check that Admin Dashboard live feed/team stats update.
11. Test pricing payment using Paystack test mode first.

14. Common Errors And Fixes

Error	Likely cause	Fix
MODULE_NOT_FOUND server.js	server.js missing or wrong start command.	Make sure server.js exists and Render start command is npm start.
Missing SUPABASE_URL...	Render environment variables not set.	Add SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, PAYSTACK_SECRET_KEY.
Backend timed out	Render service sleeping, failed deploy, or start crash.	Open Render logs and check the first red error. Redeploy after fixing.
CORS error	Vercel URL not in allowedOrigins or has wrong slash.	Add exact origin: https://use-sales-tracker.vercel.app.
Team service unavailable	Frontend cannot reach backend.	Check Render health URL and frontend TEAM_API_URL.
Profile not found	User exists in auth but no profiles row.	Create/fix the profile row in Supabase.
Payment succeeds but access not active	Webhook not set or service key wrong.	Set Paystack webhook URL and confirm backend logs receive charge.success.

15. What A Good Deployment Looks Like

Render: latest deploy says Live, and `/api/health` returns `{"ok":true}`.

Vercel: latest deploy says Ready, and the website opens without local backend messages.

Supabase: new users have profiles, businesses, and subscription data.

Paystack: webhook delivery shows successful HTTP 200 responses.

16. Simple Deployment Flow To Remember

1. Finish code locally.
2. Test locally.
3. Push to GitHub.
4. Deploy frontend to Vercel.
5. Deploy backend to Render.
6. Add Render environment variables.
7. Set frontend API URLs to Render backend.
8. Set Paystack webhook to Render backend.
9. Test health, login, products, reps, sales, payments.

Best habit: when deployment fails, do not guess. Open the Render or Vercel logs, copy the first red error, and fix that exact issue first.